Database-Native Persistence Semantics for Agent Memory

Workshop-Style Draft

NornicDB Project

May 2026

Abstract

Agent memory systems often conflate durable facts, episodic memories, behavioral directives, and inference-time reasoning. This causes factual knowledge to decay, memories to persist indefinitely, or behavioral rules to be promoted from correlated evidence. We present NornicDB, a graph database substrate that exposes persistence semantics as database-native primitives: canonical fact versioning with temporal validity windows, append-only mutation logs with WAL receipts, a profile-and-policy-driven scoring subsystem that gates retrieval visibility, and a two-layer anti-sycophancy defense combining session-aware gating with Kalman-filtered behavioral signals. All primitives – the canonical graph ledger, MVCC version resolution, the mutation log, the scoring subsystem, and the declarative Cypher DDL extensions for decay bundles, decay bindings, promotion profiles, and promotion policies – are implemented and shipped in NornicDB v1.1.0. The decay surface supports forward Ebbinghaus forgetting curves and an optional database-level inverted consolidation curve through a single negative-half-life signal, composed with a `LAST_ACCESSED` anchor and `ON ACCESS` metadata updates so that idle time strengthens the score and retrieval resets it. This inverted curve is a NornicDB policy primitive inspired by consolidation literature and compatible with Roynard's layer separation, not a mechanism explicitly named in Roynard's paper. We show how this design addresses requirements raised by recent work on temporal GraphRAG [1], enterprise replayable memory [2], and missing knowledge layers in cognitive architectures [3]. The contribution is not a new memory policy, but a storage and query substrate capable of expressing multiple memory policies without hardcoding cognitive categories.

Keywords: agent memory; graph databases; temporal validity; persistence semantics; policy-driven retrieval; anti-sycophancy.

---

1. Introduction

A gap has opened between the sophistication of agent memory research and the persistence guarantees those systems actually provide. Three independent lines of recent work expose the same underlying problem from different angles.

Han et al. [1] show that retrieval-augmented generation systems fail on time-sensitive facts because they lack temporal representations – the same entity at different times is indistinguishable in a vector embedding. Their Temporal GraphRAG (TG-RAG) proposal introduces timestamped graph edges and hierarchical time summaries, but the temporal modeling lives in the application layer, not in the storage substrate.

Srinivasan [2] argues that enterprise deployment of decision agents is blocked not by decision accuracy but by four systems properties: deterministic replay, auditable rationale, multi-tenant isolation, and stateless horizontal scale. Her Deterministic Projection Memory (DPM) architecture treats the trajectory as an append-only event log and performs a single task-conditioned projection at decision time. The insight is that append-only immutability and replay are load-bearing requirements that stateful memory architectures violate by construction.

Roynard [3] identifies a category error in existing cognitive architectures: systems apply cognitive decay to factual claims, or treat facts and experiences with identical update mechanics. His four-layer decomposition (Knowledge, Memory, Wisdom, Intelligence) assigns each layer fundamentally different persistence semantics – indefinite supersession, Ebbinghaus decay, evidence-gated revision, and ephemeral inference – and argues that no current framework or system provides this separation at the storage level. Roynard explicitly critiques NornicDB's earlier three-tier decay model, noting that "a paper's findings do not become less true after 69 days." The critique is well-taken: it identifies a category error where the conflation of "I have not accessed this recently" with "this is less valuable" produces semantically wrong retrieval behavior.

These three papers converge on a shared diagnosis: the storage layer matters. Temporal validity, append-only provenance, layer-specific decay, evidence-gated promotion, and replay surfaces are not application concerns that can be bolted on after the fact. They are persistence semantics that belong in the database substrate.

NornicDB is a graph database designed for AI agent memory. Rather than implementing a particular cognitive architecture, it exposes persistence semantics as database-native primitives. The canonical graph ledger provides fact versioning and temporal validity. The mutation log and WAL receipts provide append-only provenance and replay. The scoring subsystem – authored through declarative Cypher DDL extensions – provides policy-driven decay and evidence-gated promotion with scoring-before-visibility semantics. The `reveal()` operator separates visibility policy from compliance retention. A two-layer anti-sycophancy defense combines session-aware gating (removing bias) with Kalman-filtered behavioral signals (removing variance), adapted from a flight-controller gyroscope filter. Together, these primitives express the requirements raised by all three lines of work without hardcoding any single cognitive model.

By *database-native persistence semantics*, we mean that validity, supersession, decay, promotion, visibility, provenance, and replay are represented as schema, constraints, indexes, query semantics, and transaction-log behavior – rather than as conventions enforced only by an application-side agent loop.

Contribution statement.

- *What is new?* A database substrate that implements temporal validity, supersession, append-only mutation history, scoring-before-visibility, layer-specific decay (including an optional inverted consolidation curve through a negative-half-life signal), and evidence-gated promotion as shipped graph-native primitives – all authored through declarative Cypher DDL rather than application-side memory code, with decay parameter bundles separated from targeted decay bindings.
- *Why does it matter?* Current agent memory systems either ignore persistence semantics entirely or implement them as ad hoc application logic, making them non-portable, non-auditable, and difficult to compose. NornicDB makes these semantics configurable per content type through the same schema-oriented mechanisms operators already use for constraints and indexes.
- *What evidence supports it?* We demonstrate that NornicDB's primitives – all of which ship in v1.1.0 [11] – address the specific requirements raised by three independent research threads, provide worked examples with concrete Cypher DDL, and describe a concrete anti-sycophancy mechanism with quantitative dampening behavior.

---

2. Related Work

We organize related work around four threads. An expanded treatment is in the companion `related-work.md`.

2.1 Temporal GraphRAG and Time-Sensitive Retrieval

TG-RAG [1] models external corpora as a bi-level temporal graph: a temporal knowledge graph with timestamped relations and a hierarchical time graph with multi-granularity summaries. The design supports incremental updates by extracting new temporal facts and merging them into the existing graph. TG-RAG demonstrates that temporal modeling significantly improves retrieval on time-sensitive questions, but implements temporal semantics in the application layer rather than as database primitives.

2.2 DPM and Event-Sourced Enterprise Memory

Srinivasan's DPM [2] treats agent memory as an append-only event log plus a single task-conditioned projection at decision time. The architecture is designed against four enterprise properties (deterministic replay, auditable rationale, multi-tenant isolation, statelessness for horizontal scale) rather than against decision-accuracy benchmarks. At tight budgets (20x compression), DPM improves factual precision by +0.52 (Cohen's h=1.17, p=0.0014) and reasoning coherence by +0.53 (h=1.13, p=0.0034) over summarization-based memory. DPM validates append-only immutability as a first-class design requirement: the replay guarantee is structural, not behavioral.

2.3 The Missing Knowledge Layer and Persistence Semantics

Roynard [3] argues that CoALA [8] and JEPA [9] both lack an explicit Knowledge layer with its own persistence semantics, producing a category error where systems apply cognitive decay to factual claims. The four-layer decomposition (Knowledge, Memory, Wisdom, Intelligence) assigns each layer a different update mechanism: supersession, Ebbinghaus decay, evidence-gated revision, and ephemeral inference. Roynard explicitly critiques NornicDB's earlier three-tier decay model – the critique motivated a full redesign of NornicDB's scoring architecture from hardcoded tiers to a declarative bundle, binding, profile,

and policy system. Roynard subsequently reviewed the redesigned architecture and assessed it as "a substantive response to the critique," noting that the four-layer profile set is "a four-different-update-mechanisms mapping, which is precisely what the paper's persistence-semantics table argues for" [10]. The redesigned subsystem ships in NornicDB v1.1.0 [11], including a negative-half-life signal that inverts any decay function family in place. When paired with a `LAST_ACCESSED` anchor, this implements a NornicDB-level consolidation-by-idleness curve: a policy option consistent with Roynard's separation of Memory from Knowledge, but not a curve explicitly defined by Roynard.

2.4 Existing Graph Memory Systems

MemGPT [4] pioneered OS-style memory management for LLMs. Mem0 [5] applies identical CRUD operations to facts and experiences. Graphiti [6] introduced bi-temporal event sourcing for graph-based agent memory. Signet applies uniform 0.95^days decay to all content types regardless of category. These systems advance the state of agent memory but do not expose persistence semantics as configurable database primitives – the update mechanics are hardcoded per system.

Summary. All four threads demonstrate that temporal and memory-aware retrieval improves agents. NornicDB asks a different question: which of those semantics should be enforced by the database substrate itself? TG-RAG, Graphiti, and GAM implement temporal semantics above the database. DPM implements replay above the database. Roynard's layer separation exists only as a conceptual framework. NornicDB makes these concerns native to the storage and query layer.

---

3. Design Requirements

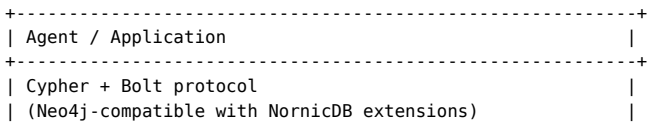
NornicDB targets eight persistence-semantic requirements drawn from the gaps identified in Section 2. Each requirement is traceable to at least one of the three research threads.

#	Requirement	Motivation
R1	**Temporal validity** – every fact carries a validity window, not just a creation timestamp	TG-RAG: same entity at different times must be distinguishable [1]
R2	**Supersession, not forgetting** – old facts are linked to their replacements, never deleted	Roynard: knowledge does not decay, it gets superseded [3]
R3	**Append-only mutation history** – every write is logged immutably	DPM: the event log is the single source of truth for replay [2]
R4	**Scoring-before-visibility** – retrieved results pass through a policy gate before reaching the agent	Roynard: retrieval should reflect persistence semantics, not just similarity [3]
R5	**Layer-specific decay** – different content types decay at different rates or not at all	Roynard: Knowledge = no decay, Memory = Ebbinghaus, Wisdom = evidence-gated [3]
R6	**Evidence-gated promotion** – behavioral directives require independent corroboration before promotion	Roynard: sycophancy concern from Section 3 citing Cheng et al. [3]
R7	**Replay and audit surfaces** – any past decision can be reconstructed from the log	DPM: deterministic replay is load-bearing for regulated deployment [2]
R8	**Tenant/session/agent context** – memory isolation across organizational boundaries	DPM: multi-tenant isolation is structural, not policy-based [2]

---

4. NornicDB Architecture

This section describes the four architectural components that address the requirements above. NornicDB is implemented in Go and uses Badger as its storage engine. It is Neo4j-compatible at the Cypher and Bolt protocol layers, with NornicDB-specific extensions for persistence semantics. All four components – the canonical graph ledger (Section 4.1), the scoring subsystem (Section 4.2), the anti-sycophancy defense (Section 4.3), and the declarative authoring surface (Section 4.4) – are implemented and shipped in NornicDB v1.1.0 [11].



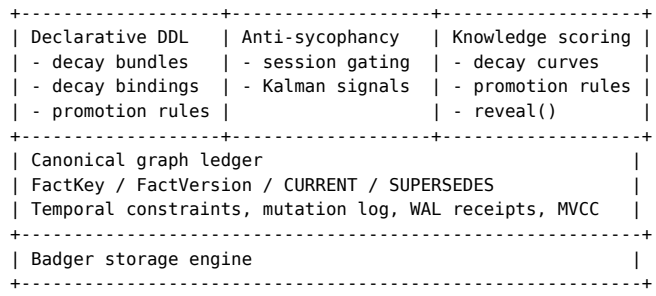


Figure 1. NornicDB architecture overview. All four components are implemented and shipped in NornicDB v1.1.0 [11].

4.1 Canonical Graph Ledger

The canonical graph ledger is the core persistence layer. It defines a reusable pattern for versioned facts using three constructs with generic placeholder labels. Operators replace these with domain-specific labels when adopting the pattern – for example, the decay profiles in Section 4.2.1 target `:KnowledgeFact`, the domain label for durable facts in the Knowledge layer, which operators structure using the constructs below.

- **`FactKey`**: a stable identifier for a factual claim (e.g., `company:acme:revenue:2024`). The key is the identity of the fact across versions.
- **`FactVersion`**: an immutable snapshot of the fact at a point in time. Each version carries a validity window (`validFrom`, `validUntil`), provenance metadata (source, timestamp, authoring agent), and the content payload.
- **`CURRENT` edge**: a cardinality-1 edge from `FactKey` to the active `FactVersion`. Supersession creates a new `FactVersion`, moves the `CURRENT` edge, and records a `SUPERSEDES` relationship from the new version to the old. The old version is never deleted. Supersession is always modeled through graph structure (`:SUPERSEDES` edges).

Temporal integrity is enforced by a `TEMPORAL NO OVERLAP` constraint on `FactVersion` nodes sharing a `FactKey`. This guarantees that at most one version of a fact is active at any point in real-world time, enabling unambiguous point-in-time queries. The constraint is checked at write time and rejects any version whose validity window overlaps an existing version for the same key.

The mutation log records every write operation (create, supersede, suppress, restore) as an append-only entry with a WAL receipt. Combined with MVCC version resolution, this provides the replay surface required by R3 and R7: any state of the graph can be reconstructed by replaying the mutation log from the beginning. Time-travel queries (`GetNodesByLabelVisibleAt`, `GetEdgesByTypeVisibleAt`) iterate MVCC version records directly with iterator pinning on the snapshot version, without touching secondary indexes.

Addressing R1, R2, R3, R7.

4.2 Knowledge-Layer Scoring

The scoring subsystem sits between storage and retrieval. When a query touches an entity, the engine resolves and applies scoring *before* deciding whether the entity is visible to the query. An entity whose final score falls below the visibility threshold does not appear in `MATCH` results, `WHERE` evaluation, or search hits unless the caller explicitly uses `reveal()`.

The subsystem is independent from – and layered on top of – the compliance retention subsystem (GDPR Art.17 erasure, legal holds, retention-policy-driven archival). `reveal()` bypasses only the scoring gate; it cannot resurrect compliance-deleted entities.

4.2.1 Decay Profiles Decay profiles define how a score evolves over time. They are authored through Cypher DDL extensions and targeted to specific node labels, edge types, or wildcards:

```
-- Decay bundle: reusable parameters, no target, inert on its own.
CREATE DECAY PROFILE memory_episode_params OPTIONS {
  halfLifeSeconds: 604800,
  function: 'exponential',
  scoreFrom: 'VERSION',
  visibilityThreshold: 0.10
}

-- Decay binding: targeted application of a bundle to a label.
CREATE DECAY PROFILE memory_episode_retention
```

```
FOR (n:MemoryEpisode)
APPLY {
  DECAY PROFILE 'memory_episode_params'
  DECAY VISIBILITY THRESHOLD 0.10
}

-- Knowledge and Wisdom use no-decay bindings; supersession or
-- revision is modeled through graph structure, not time decay.
CREATE DECAY PROFILE knowledge_fact_retention
FOR (n:KnowledgeFact)
APPLY { NO DECAY }

CREATE DECAY PROFILE wisdom_directive_retention
FOR (n:WisdomDirective)
APPLY { NO DECAY }
```

The implementation supports four decay-function families – `exponential` (Ebbinghaus), `linear`, `step`, and `none` – composed with a sign modifier on `halfLifeSeconds`. A positive half-life produces the standard forgetting curve; a negative half-life inverts the function in place, so the compiled score becomes `1 - f(age, |halfLife|)`. The exponential forward and inverse curves are:

```
forward (positive halfLife):   score(t) = exp(-t x ln(2) / halfLife)
inverse (negative halfLife):   score(t) = 1 - exp(-t x ln(2) / |halfLife|)
```

The inversion is purely a curve property and composes with every function family and every score-start anchor; it is not a separate function constant or modifier flag in the schema. Combined with the `LAST_ACCESSED` anchor (below), the inverse exponential implements an optional consolidation-by-idleness curve in which a memory grows stronger while idle and is reset on access. This is NornicDB's policy mechanism rather than a named curve from Roynard's paper.

Score-start selection. Each decay profile declares which timestamp drives decay age through a `scoreFrom` option:

- `CREATED` – decay age begins at the entity's original creation timestamp. Updates do not reset decay.
- `VERSION` – decay age begins at the latest visible MVCC version timestamp. Updates reset the decay clock. Combined with validity windows, this provides bi-temporal scoring: the MVCC system determines *which* version is visible, and `scoreFrom: 'VERSION'` determines *how old* that version appears for scoring purposes.
- `CUSTOM` – decay age begins at a user-specified property value, enabling domain-specific anchors.
- `LAST_ACCESSED` – decay age begins at the entity's last access timestamp recorded by the access metadata index, falling back to `CREATED` until an access is observed. This mode only reads access metadata; a promotion policy on the same target must write `lastAccessedAt` in its `ON ACCESS` block. With a negative half-life it models NornicDB's optional consolidation-by-idleness policy: idle time grows the score; access resets the anchor and drops the score back toward zero.

The scoring timestamp ("now") is not `time.Now()` – it is the transaction's MVCC snapshot timestamp, passed through the scorer as `scoringTime`. This ensures deterministic, repeatable scoring within a transaction: the same entity queried twice in the same transaction returns the same score.

`scoreFloor` versus `visibilityThreshold`. Decay profiles expose two independent levers that are easy to conflate. `scoreFloor` is a *score-value clamp* (the last `max()` in the score pipeline); `visibilityThreshold` is a *suppression cutoff* (`finalScore < visibilityThreshold` hides the entity). They are independent – setting `scoreFloor` alone does not keep an entity visible unless the floor itself is at or above the threshold. The distinction matters most on inverse curves, where the curve evaluates to exactly `0.0` immediately after access: a profile that wants the entity to remain visible right after access must set `scoreFloor >= visibilityThreshold`.

Property-level and edge-level targeting. Decay is not limited to whole nodes. Individual properties can receive their own decay rates (e.g., `n.lastConversationSummary` decays with a 30-day half-life while `n.tenantId` is `NO DECAY`). Edges are first-class decay targets – a `CO_ACCESSED` edge can decay independently of its endpoint nodes. Properties participating in structural indexes (lookup, range, composite) are designated immune to decay scoring, ensuring stable aggregation and joins. Vector and fulltext indexes do not confer immunity – property-level decay can exclude content from embeddings without affecting Cypher queryability.

4.2.2 Three-Tier Scoring Optimization The resolution cascade is pre-compiled at DDL time into a compiled binding table – a direct `map[string]*CompiledBinding` keyed by sorted label set or edge type. Resolution at query time is a single map lookup, not a cascade.

```
  Query touches entity
         |
```

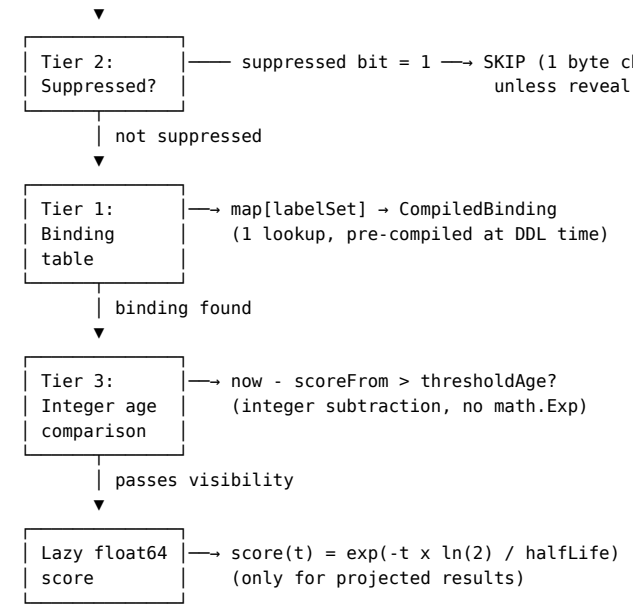


Figure 2. Three-tier scoring optimization. Each tier is cheaper than the next; most entities are resolved without computing a float64 score.

Three optimization tiers keep the hot path fast:

1. **Tier 1 – Compiled binding table.** One map lookup per entity to find the applicable decay profile and promotion policy. Rebuilt on DDL change (rare).
2. **Tier 2 – Suppressed-bit fast path.** Suppressed entities carry a persisted `VisibilitySuppressed` boolean. The read path checks this bit (one byte) before any profile resolution. If suppressed and no `reveal()`, skip immediately. To preserve correctness across label changes, `UpdateNode` re-evaluates suppression under the entity's new labels and rewrites the persisted flag – without this rebind, an entity moved out of a suppressing binding would stay suppressed forever despite the curve permitting visibility under the new label set.
3. **Tier 3 – Integer age comparison.** For monotonic forward decay, pre-compute a threshold age: `thresholdAge = -halfLife x ln(visibilityThreshold) / ln(2)`. At read time, compare `now - scoreFrom > thresholdAge` using integer subtraction on UnixNano values – no `math.Exp()` needed for the visibility check. The precise float64 score is computed lazily only when the entity survives visibility and is projected into results. Inverse curves bypass the threshold-age fast path because they are not monotonically decreasing in age; they always go through the full score calculation, which costs slightly more CPU per read in exchange for the consolidation semantics.

4.2.3 Promotion Policies Promotion policies contain the logic that determines an entity's promotion tier. They are separate objects from decay bindings: decay computes a base score, an optional matching promotion policy contributes a multiplier/floor/cap, and the two are composed into a final score:

```
t            = now - anchor
baseScore    = baseDecay(t)
multiplier   = matchedPromotion.multiplier if matched else 1.0
promoted     = baseScore * multiplier
clampedPromo = min(promoCap, max(promoFloor, promoted))
finalScore   = max(decayBindingFloor, clampedPromo)
suppressed   = finalScore < visibilityThreshold
```

Promotion policies include `WHEN` predicates and optional `ON ACCESS` mutation blocks:

```
CREATE PROMOTION POLICY session_record_tiering
FOR (n:SessionRecord)
APPLY {
  ON ACCESS {
    SET n.accessCount = coalesce(n.accessCount, 0) + 1
    SET n.lastAccessedAt = timestamp()
  }

  WHEN n.accessCount >= 3
    APPLY PROFILE 'reinforced_tier'
```

```
  WHEN n.accessCount >= 5 AND n.sourceAgreement >= 0.95
    APPLY PROFILE 'canonical_tier'
}
```

A critical design decision: `ON ACCESS` mutations execute *only if the entity passes the visibility gate*. Suppressed entities do not accumulate access state – this prevents suppressed entities from recording "accesses" that occurred only because the scorer was evaluating them, not because a user or query actually retrieved them.

`ON ACCESS` mutations write exclusively to a separate `accessMeta` index (Badger key prefix `0x11`), never to the node or edge itself. Nodes and edges remain read-only during policy evaluation. The `policy()` Cypher function exposes accessMeta for diagnostics (e.g., `policy(n).accessCount`).

Hot-path performance. `ON ACCESS` increments are accumulated in-process via per-P (per-processor) sharded counter rings using `sync.Pool` for P-local affinity, targeting zero cross-core contention regardless of graph topology. Each shard holds a delta, not an absolute value – no msgpack serialization, no Badger write, no allocation on the read path. A background flush goroutine drains the counter ring on a configurable interval (default 2s) and applies a single batched Badger write. Access counts are eventually consistent with a bounded lag of one flush interval.

4.2.4 Deindexing When a node or edge becomes visibility-suppressed, it must be removed from secondary indexes. The deindexing mechanism avoids expensive full-index scans through a per-entity `IndexEntryCatalog` that records the exact Badger keys written for each entity across all index types. A background deindex job reads the catalog and performs blind batched deletes against the exact keys. Presence-marker tombstones live under a dedicated prefix (`0x17`) – zero-length entries whose existence tells current-time index scans to skip the candidate. Time-travel queries are unaffected because they iterate MVCC version records directly and never touch secondary indexes.

Addressing R4, R5.

4.3 Two-Layer Anti-Sycophancy Defense

The Wisdom layer requires evidence-gated promotion (R6). When an LLM evaluates a memory – "how relevant is this?" or "how confident are you in this fact?" – the answer can hallucinate. A mediocre memory might get scored 0.99 confidence because the model was being agreeable (sycophancy), confused, or having a bad inference. Storing that score directly would allow a single hallucinated spike to promote garbage to the canonical knowledge layer.

The defense addresses this with two layers that handle different failure modes:

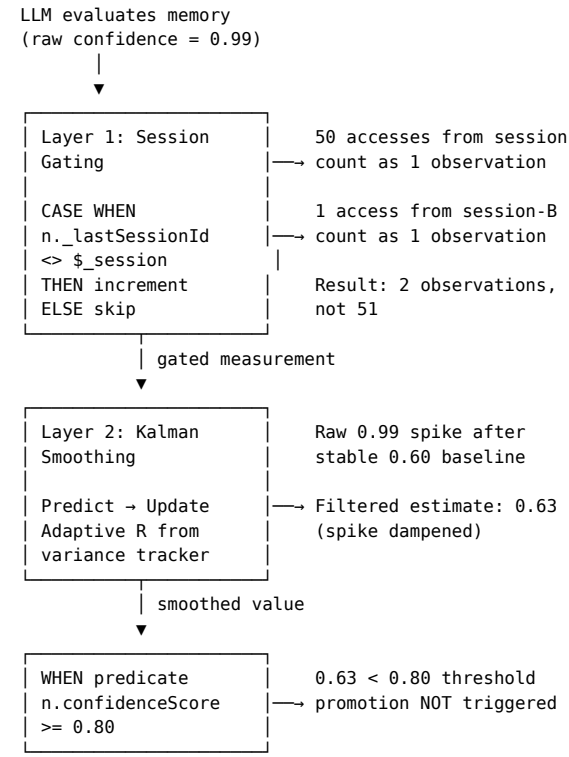


Figure 3. Two-layer anti-sycophancy defense. Session gating removes bias (same-session repetition). Kalman smoothing removes variance (hallucinated spikes). The promotion predicate sees the filtered value, not the raw measurement.

Layer 1: Session-Aware Gating (Removes Bias) LLM-driven access patterns are not just noisy – they can be systematically biased. If the same agent accesses the same memory 50 times in one session (a sycophancy loop), those are not 50 independent observations. They are one observation repeated.

The engine projects query context variables from HTTP request headers (`X-Query-Session` → `$_session`, `X-Query-Agent` → `$_agent`, `X-Query-Tenant` → `$_tenant`, and any `X-Query-<Key>` → `$_<key>`). These are read-only, ephemeral, and generic – the engine does not decide which context matters. Operators define their own gating logic:

```
ON ACCESS {
  -- Only count new sessions. Same session = same observation.
  WITH KALMAN SET n.crossSessionRate =
    CASE WHEN n._lastSessionId <> $_session
      THEN coalesce(n.crossSessionRate, 0) + 1
      ELSE n.crossSessionRate
    END
  SET n._lastSessionId = $_session
}
```

With this gating, 50 accesses from one session count as 1. Only genuinely distinct sessions contribute to the cross-session rate.

Layer 2: Kalman Smoothing (Removes Variance) After gating removes bias, the remaining noise in behavioral signals is handled by a scalar Kalman filter. The algorithm is adapted from the imu-f flight controller's gyroscope filter – the same problem of preventing transient sensor spikes from corrupting estimated state.

The filter implementation:

1. **Velocity-based prediction**: `x += (x - lastX)` projects the state forward. A genuine trend continues; a hallucinated spike deviates from the trajectory.
2. **Error-boosted covariance**: `p = p + q x e`, where `e = |1 - target/lastX|`. Far from the expected state, prediction uncertainty grows.
3. **Gain-limited measurement update**: `K = p / (p + R)`. When R is high (noisy measurements), K is low, and a single wild measurement barely moves the estimate.
4. **Adaptive R via variance tracking** (auto mode): R is continuously recalculated as `sqrt(sample_variance) x VARIANCE_SCALE` over a sliding window (default 32 samples), ported directly from the flight controller's `update_kalman_covariance`. As measurements become noisier, R increases and the filter dampens harder.

Three modes are exposed through the `WITH KALMAN` Cypher modifier:

Syntax	Behavior
`WITH KALMAN`	Auto mode – R self-adjusts based on observed measurement variance
`WITH KALMAN{q: 0.05, r: 50.0}`	Manual mode – fixed Q and R
`WITH KALMAN{q: 0.05}`	Hybrid – user-set Q, R self-adjusts

Concrete dampening behavior. The following trace shows the filter's response to a hallucinated confidence spike on a memory episode with a stable true confidence around 0.60:

Access	Raw Confidence	Kalman-Filtered	Note
1	0.60	0.60	First measurement, filter initializes
2	0.62	0.61	Consistent, filter tracks smoothly
3	0.58	0.60	Small dip, barely changes estimate
4	0.61	0.60	Stable around 0.60
5	**0.99**	**0.63**	**Hallucination. K is low because the signal has been stable – spike barely moves estimate**
6	0.59	0.62	Back to normal, filter recovers

Access	Raw Confidence	Kalman-Filtered	Note
7	0.61	0.62	Tracking the real value again

Without the Kalman filter, access 5 would have set `confidenceScore = 0.99` and triggered high-confidence promotion on a mediocre memory. With it, the estimate barely budged. Critically, a *sustained* sequence of high-confidence observations from genuinely independent sources would eventually move the estimate upward – the filter dampens spikes but does not suppress genuine signals.

What to filter and what not to filter. Kalman smoothing is appropriate for derived behavioral metrics where the input signal has genuine measurement noise: confidence scores from LLM evaluation, relevance assessments, cross-session access rates, agreement ratios. It is *not* appropriate for raw monotonic counters (`accessCount` – always goes up by 1, no noise) or timestamps (`lastAccessedAt` – a point in time, no noise). These use plain `SET` without `WITH KALMAN`.

Kalman filter state (estimate, gain, covariance, observation count, variance window) is persisted per-property in a typed `KalmanFilters map[string]*KalmanPropertyState` field on `AccessMetaEntry`, and is inspectable through the `policy()` Cypher function for diagnostics:

```
MATCH (m:MemoryEpisode {id: $id})
RETURN policy(m).kalmanFilters.confidenceScore.filteredValue AS smoothedConfidence,
       policy(m).kalmanFilters.confidenceScore.filter.k AS kalmanGain,
       policy(m).kalmanFilters.confidenceScore.variance.v AS currentVariance
```

Addressing R6.

4.4 Declarative Authoring Surface

All persistence semantics are authored through Cypher DDL extensions, following the same schema-oriented patterns NornicDB uses for constraints and indexes. The current catalog exposes four object kinds. Two are inert parameter packages, and two are targeted bindings with a `FOR` clause:

Object kind	DDL form	Has target?	Role
Decay bundle	`CREATE DECAY PROFILE <name> OPTIONS { ... }`	No	Names reusable decay parameters.
Decay binding	`CREATE DECAY PROFILE <name> FOR (...) APPLY { ... }`	Yes	Activates decay scoring for matched labels or edge types.
Promotion profile	`CREATE PROMOTION PROFILE <name> OPTIONS { ... }`	No	Names multiplier/floor/cap parameters.
Promotion policy	`CREATE PROMOTION POLICY <name> FOR (...) APPLY { ... }`	Yes	Tracks access and/or applies promotion profiles when `WHEN` predicates match.

The overload on `CREATE DECAY PROFILE` is deliberate: with `OPTIONS` it creates a bundle; with `FOR (...) APPLY` it creates a binding. `FOR` is the only targeting mechanism and is used only by decay bindings and promotion policies, not by inert bundles or promotion profiles.

Binding resolution is deterministic rather than exclusively one-definition-only. Multi-label targets take precedence over single-label targets; exact label matches take precedence over wildcard targets; if two definitions have equal specificity, the resolver emits a diagnostic and uses the first registered binding. Promotion-policy resolution follows the same specificity rules.

The supported operational surface is Cypher-native:

```
CALL nornicdb.knowledgepolicy.info();
CALL nornicdb.knowledgepolicy.profiles();
CALL nornicdb.knowledgepolicy.policies();
CALL nornicdb.knowledgepolicy.resolve('', 'MemoryEpisode', '');
CALL nornicdb.knowledgepolicy.deindexStatus();
```

```
SHOW DECAY PROFILES;
SHOW PROMOTION PROFILES;
SHOW PROMOTION POLICIES;
```

The scoring subsystem is gated by configuration (`memory.decay_enabled`). When disabled, profiles and policies may remain in the schema, but runtime scoring and suppression are neutral: entities score `1.0`, unmatched labels also score `1.0`, and suppression does not apply.

4.4.1 Built-in Ebbinghaus-Roynard Bootstrap For a new namespace with decay enabled and no existing knowledge-policy schema entries, NornicDB can seed an Ebbinghaus-Roynard bootstrap. The bootstrap installs defaults for the Knowledge, Memory, and Wisdom persistence layers: `:KnowledgeFact` and `:WisdomDirective` are bound to no-decay semantics, `:MemoryEpisode` receives a 7-day exponential profile, evidence edges receive 30-day decay, and provenance edges such as `:SUPERSEDES`, `:CONSOLIDATES_TO`, `:REVISES`, and `:DERIVED_FROM` remain permanent. The bootstrap is intentionally conservative: it does not overwrite existing policy schema, and the canonical graph ledger layer is installed separately so that operators can tailor fact-key/version labels to their domain.

Addressing R4, R5, R8.

---

5. Worked Examples

5.1 Fact Superseded by a Later Fact

An agent ingests a research paper stating "LoRA achieves 95% of full fine-tuning quality at 0.1% of parameters." This creates a `FactKey` with a single `FactVersion`:

```
CREATE (k:FactKey {id: "method:lora:quality_ratio"})
CREATE (v:FactVersion {value: "95% at 0.1%", validFrom: date("2024-01-15"), source: "paper_A"})
CREATE (k)-[:CURRENT]->(v)
```

Six months later, a new paper reports "DoRA achieves 97% at 0.08%." NornicDB creates a new version, moves `CURRENT`, and links the old version:

```
MATCH (k:FactKey {id: "method:lora:quality_ratio"})-[c:CURRENT]->(old:FactVersion)
CREATE (new:FactVersion {value: "97% at 0.08%", validFrom: date("2024-07-20"), source: "paper_B"})
SET old.validUntil = date("2024-07-20")
DELETE c
CREATE (k)-[:CURRENT]->(new)
CREATE (new)-[:SUPERSEDES]->(old)
```

A query for the current state returns only the DoRA fact. An `AS OF` query returns the version valid at that time. Neither fact is decayed or deleted. This example uses the canonical graph ledger's generic labels (`:FactKey`, `:FactVersion`); in a deployment using the Ebbinghaus-Roynard model (Section 4.2.1, Section 5.2-5.3), operators would use `:KnowledgeFact` as their domain label, structured using the same pattern. The decay profile for `:KnowledgeFact` is `NO DECAY` – these entities are always visible with score 1.0.

5.2 Memory Episode Decaying Unless Reinforced

A user mentions preferring dark mode during a conversation. This creates a `:MemoryEpisode`. The decay profile and promotion policy for this label:

```
CREATE DECAY PROFILE memory_episode_params OPTIONS {
  halfLifeSeconds: 604800,
  function: 'exponential',
  scoreFrom: 'LAST_ACCESSED',
  visibilityThreshold: 0.10
}

CREATE DECAY PROFILE memory_retention
FOR (n:MemoryEpisode)
APPLY {
  DECAY PROFILE 'memory_episode_params'
  DECAY VISIBILITY THRESHOLD 0.10
}
```

```
CREATE PROMOTION POLICY memory_reinforcement
FOR (n:MemoryEpisode)
APPLY {
  ON ACCESS {
    SET n.accessCount = coalesce(n.accessCount, 0) + 1
    SET n.lastAccessedAt = timestamp()

    WITH KALMAN{q: 0.05, r: 50.0} SET n.confidenceScore = $evaluatedConfidence
  }

  WHEN n.accessCount >= 3
    APPLY PROFILE 'reinforced'

  WHEN n.accessCount >= 5 AND n.confidenceScore >= 0.8
    APPLY PROFILE 'high_confidence'
}
```

This example uses `scoreFrom: 'LAST_ACCESSED'` to demonstrate access-based reinforcement: the promotion policy's `ON ACCESS` block writes `lastAccessedAt`, and the decay binding reads that metadata as the next anchor. After 7 days without reinforcement, `score = exp(-7d x ln2 / 7d) = 0.5`. After 21 days, `score = 0.125`, still above the visibility threshold of 0.10; the crossing occurs after approximately 23.25 days, and at 24 days the score is about 0.093, so the episode is suppressed from ordinary retrieval. If the user mentions dark mode again before suppression, the episode is accessed, `lastAccessedAt` is refreshed in access metadata, and its confidence score is Kalman-smoothed. A deployment that instead uses the default `scoreFrom: 'VERSION'` Memory profile resets the decay clock only on a node version update, not merely on `ON ACCESS` metadata writes.

Note: "User prefers dark mode" as a *verified preference* (not an episodic observation) would be stored as a `:KnowledgeFact` with `NO DECAY` and supersession semantics. The distinction between "the user said this once" (Memory) and "this is a verified preference" (Knowledge) is exactly the layer separation Roynard [3] argues for.

5.3 Wisdom Directive with Anti-Sycophancy

An agent observes across sessions that checking the user's theme preference before answering UI questions leads to higher satisfaction. A wisdom directive is created with the following policy:

```
CREATE DECAY PROFILE wisdom_retention
FOR (n:WisdomDirective)
APPLY { NO DECAY }

CREATE PROMOTION POLICY wisdom_stability
FOR (n:WisdomDirective)
APPLY {
  ON ACCESS {
    SET n.evaluationCount = coalesce(n.evaluationCount, 0) + 1

    -- Cross-session evidence: gated by session, then Kalman-smoothed
    WITH KALMAN SET n.crossSessionSupport =
      CASE WHEN n._lastSessionId <> $_session
        THEN coalesce(n.crossSessionSupport, 0) + 1
        ELSE n.crossSessionSupport
      END
    SET n._lastSessionId = $_session

    -- LLM confidence: noisy behavioral signal, Kalman-smoothed
    WITH KALMAN{q: 0.05, r: 50.0} SET n.confidenceScore = $evaluatedConfidence
  }

  WHEN n.crossSessionSupport >= 5 AND n.confidenceScore >= 0.75
    APPLY PROFILE 'established_wisdom'

  WHEN n.crossSessionSupport >= 3
    APPLY PROFILE 'provisional_wisdom'
}
```

If a single agent in a single session accesses the directive 50 times, `crossSessionSupport` remains at its initial value because `$_session` matches `n._lastSessionId` on every access after the first – the `CASE WHEN` gate prevents same-session

repetition from inflating the evidence count. Even if the LLM hallucinates a confidence spike of 0.99, the Kalman filter dampens it to approximately 0.63 (see Section 4.3), preventing accidental promotion.

Only genuinely independent observations from distinct sessions increment the evidence counter. This is the anti-sycophancy property: repetition is not evidence, and noise is not signal.

---

6. Comparison to Research Requirements

Requirement from literature	TG-RAG / DPM / Roynard concern	NornicDB mechanism	Status
Time-sensitive facts	Timestamped relations, evolving knowledge [1]	Temporal graph + MVCC + validity windows	Implemented
Incremental updates	Avoid full reindex/rebuild [1]	Mutation log + exact index-entry catalogs	Implemented
Deterministic replay	Event log as source of truth [2]	WAL txlog + receipts	Implemented
Auditable rationale	Regulator/audit inspection of decisions [2]	Mutation log + provenance metadata per FactVersion	Implemented
Multi-tenant isolation	Structural, not policy-based isolation [2]	Query context variables from `X-Query-Tenant` header	Implemented
Knowledge should not decay	Critique of Ebbinghaus applied to facts [3]	`NO DECAY` profile, supersession edges	Implemented
Memory should decay	Episodic observations fade unless reinforced [3]	Ebbinghaus decay profiles with configurable half-life	Implemented
Memory consolidation	Memories decay unless consolidated; cognitive literature includes interference, reconsolidation, and sleep-dependent consolidation [3]	Ebbinghaus decay plus optional negative-half-life curves with `scoreFrom: 'LAST_ACCESSED'`	Implemented as a NornicDB policy primitive
Evidence-gated wisdom	Sycophancy concern: access != evidence [3]	Session gating + Kalman-smoothed behavioral signals	Implemented
Bi-temporal event sourcing	Four-timestamp model for temporal conflicts [3, 6]	`scoreFrom: 'VERSION'` + MVCC snapshots + validity windows	Implemented
Compliance vs. scoring separation	Legal deletion != retrieval suppression [3]	`reveal()` bypasses scoring; compliance deletion is permanent	Implemented

---

7. Evaluation Plan

A full benchmark comparison is future work. For this paper, we propose a small reproducible evaluation covering seven properties:

1. **Correctness of supersession queries.** Given N fact keys each with K sequential versions (N in {100, 1000, 10000}, K in {2, 5, 10, 50}), verify that `CURRENT` always points to the latest version and `AS OF` queries return the correct version for each validity window. Pass criteria: 100% correctness.
2. **Temporal no-overlap constraint.** Attempt to create overlapping validity windows for the same `FactKey`. Verify rejection. Verify non-overlapping versions for the same key and overlapping versions for different keys are both accepted.
3. **Replay / audit reconstruction.** Execute M operations (M in {100, 1000, 10000}; distribution: 50% create, 30% supersede, 10% suppress, 10% restore). Replay the mutation log from empty state. Verify byte-level match at each checkpoint.
4. **Suppression / reveal behavior.** Verify that scored-below-threshold entities are absent from default queries and present in `reveal()` queries, and that compliance-deleted entities are absent from both.
5. **Latency overhead of scoring-before-visibility.** Measure overhead across graph sizes S in {1K, 10K, 100K, 1M} entities. The scoring pipeline adds a per-entity integer comparison (Tier 3 fast path) – target: < 2x overhead at p95 for result sets <= 100 entities.

6. **Kalman smoothing robustness.** Feed N baseline observations at confidence 0.5, then a single spike at 0.99. Verify post-spike estimate < promotion threshold (expected: ~ 0.63). Feed sustained high-confidence observations from independent sources and verify the estimate eventually crosses the threshold. The filter must dampen spikes without suppressing genuine signals.
7. **Policy-schema validation.** Create decay bundles, decay bindings, promotion profiles, and promotion policies in the recommended authoring order. Verify catalog row shapes with `SHOW ...` and `CALL nornicdb.knowledgepolicy.profiles()`, verify effective resolution with `CALL nornicdb.knowledgepolicy.resolve(...)`, and verify runtime behavior with `decayScore()`, `decay()`, and `policy()`.

---

8. Limitations

We are candid about what NornicDB does not do.

- **NornicDB is not a cognitive architecture.** It is a database substrate. It does not decide what counts as Knowledge versus Memory versus Wisdom. That classification is the responsibility of the application layer or an upstream cognitive framework. The labeling of entities (`:KnowledgeFact`, `:MemoryEpisode`, `:WisdomDirective`) is an operator decision – the engine treats them as labels with associated decay profiles and promotion policies.
- **Application-layer consolidation decides truth.** NornicDB provides the storage primitives for supersession, but the decision that Fact B supersedes Fact A must come from an external process – the database does not evaluate factual claims. The Heimdall plugin serves as a reference implementation for this consolidation logic.
- **Evidence extraction quality depends on upstream agents.** The anti-sycophancy guarantee (access != evidence) is only as strong as the upstream process that determines what constitutes independent evidence. If that process treats correlated observations (the same reasoning chain re-touching the same directive through correlated paths) as independent, the sycophancy concern returns through the back door, as Roynard notes in his review [10]. The architectural skeleton is sound; the operational instantiation is where the failure mode would appear.
- **Kalman smoothing handles noisy behavioral signals, not truth.** The Kalman filter is a behavioral signal smoother adapted from a flight-controller gyroscope filter. It stabilizes derived metrics where the input is noisy. It does not evaluate whether a directive is actually correct. Labels representing canonical facts should use `NO DECAY` and should either have no promotion policy with `WITH KALMAN` mutations, or have a promotion policy with a neutral multiplier – truth-immune labels derive relevance from semantic matching, not access-pattern promotion.
- **The built-in bootstrap is not a full cognitive deployment.** The Ebbinghaus-Roynard bootstrap installs the scoring defaults for Knowledge, Memory, and Wisdom labels, but it intentionally excludes the canonical graph ledger bootstrap. Operators still choose domain labels, fact-key/version layout, and consolidation logic.
- **Full benchmark comparisons are future work.** This paper demonstrates architectural coverage of the requirements. The substrate primitives ship in NornicDB v1.1.0 [11]; comparative evaluation against MemGPT, Mem0, Graphiti, and other systems on shared benchmarks (e.g., BEAM [3]) is planned but not yet complete.

References

[1] J. Han, A. Cheung, Y. Wei, Z. Yu, X. Wang, B. Zhu, and Y. Yang, "RAG Meets Temporal Graphs: Time-Sensitive Modeling and Retrieval for Evolving Knowledge," arXiv:2510.13590, October 2025.

[2] V. Srinivasan, "Stateless Decision Memory for Enterprise AI Agents," arXiv:2604.20158, April 2026.

[3] M. Roynard, "The Missing Knowledge Layer in Cognitive Architectures for AI Agents," arXiv:2604.11364, April 2026.

[4] C. Packer et al., "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, October 2023.

[5] Mem0, "Mem0: The Memory Layer for AI Agents," https://github.com/mem0ai/mem0, 2024.

[6] Zep, "Graphiti: Build Real-Time Knowledge Graphs for AI Agents," https://github.com/getzep/graphiti, 2024.

[7] Y. Wang et al., "Graph Agent Memory: Heterogeneous Graph-Based Memory for LLM Agents," 2024.

[8] T. Sumers, S. Yao, K. Narasimhan, and T. Griffiths, "Cognitive Architectures for Language Agents," TMLR, 2024.

[9] Y. LeCun, "A Path Towards Autonomous Machine Intelligence," Open Review, 2022.

[10] M. Roynard, response to NornicDB knowledge-layer-persistence proposal, April 2026.

[11] NornicDB project, "v1.1.0 Release Notes," NornicDB repository, May 2026.